The Medium Constrains the Surface, Not the Representation: Variable Roles Across Programming-Language Lineages

Anonymous Author(s)
Affiliation
Address
email

Abstract

If language is a design axis rather than a passive carrier, then changing the language should change what a model can do. We test this directly in code, where the same problem can be written in several languages whose relatedness is fixed independently of our results, by their syntactic descent. Fitting a variable-role classifier on one language and evaluating it on another, we find that a model-free character n-gram baseline with the variable's own name masked is strongly constrained by language relatedness: macro-F1 0.952 between JavaScript and PHP, falling to 0.785 whenever Python is involved. A linear probe on a code model's residual stream moves 0.006 across the same boundary. The probe is not reading the input: an untrained network of identical architecture reaches 0.575, and both trained models clear it in all 18 transfer cells. The general-purpose base model that the code model is continued from is almost as invariant (-0.011), so code-*specialised* pretraining is not the cause. The surface fails not because its discriminative n-grams are missing from the target (70% of weighted mass survives, $r = +0.09$ with transfer) but because they point the other way: the share whose sign flips predicts transfer at $r = -0.98$. The medium governs surface generalisation and leaves the learned role representation largely untouched, a dissociation a behavioural evaluation would report as one degraded number.

1 Introduction

The premise of treating language as a design axis is that the medium is not neutral: rephrasing a problem changes what a model does with it. Programming languages make this testable in a way natural languages rarely do. The same algorithm can be written in Python, JavaScript, and PHP with the semantics held fixed, the surface form varying along the C-family/off-side-rule split that separates them, and the corpora aligned problem by problem. If the medium constrains what a model represents, code is where that constraint can be measured instead of argued.

A *variable role* (accumulator, loop iterator, index key) is a property of what a variable does, not of what it is called or which language it appears in, so roles are a case where the medium ought in principle to be irrelevant. Whether it is irrelevant *to a model* is empirical, and it separates two things a single accuracy number conflates: how far a surface regularity carries across languages, and how far a learned representation does.

We contribute a controlled measurement of that separation and a mechanism for the part that fails. Over all six ordered pairs of Python, JavaScript, and PHP, a model-free surface baseline tracks

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

language relatedness closely while a linear probe on the residual stream does not. An untrained network of identical architecture separates the contribution of the trained network from that of our inputs, and the general-purpose base model the code model is continued from shows it does not require code specialisation. Finally, the surface baseline fails for a reason that is not the obvious one: its features are present in the target and carry the opposite sign.

2 Setup

Corpus and alignment. XLCoST [Zhu et al., 2022] provides parallel solutions to competitive-programming problems in several languages, keyed by a problem identifier that is stable across languages. Every comparison below is *matched*: a classifier trained on language A and evaluated on language B sees only the problems that appear in both, so a drop in transfer cannot be attributed to a change of subject matter. Matching removes subject matter as an explanation, and it is also what restricts the study to three languages. We computed the full pairwise overlap (Appendix A): Python/JavaScript/PHP share $2{,}953$, $1{,}529$, and $1{,}145$ problems, and *no* other pair in XLCoST reaches 500, including same-family pairs such as C# and Java, which share 175. The three-language limit is a property of the benchmark, not a choice.

Roles and occurrences. We extract per-occurrence labels for accumulator, iterator, and index_key from language-specific parsers, cap each role at $3{,}000$ occurrences sampled as whole problems, and use a frozen hash-based split that assigns a problem to the same fold in every language. Test folds hold $1{,}377$–$3{,}341$ occurrences with smallest-class counts of 138–$1{,}645$, so one occurrence of the smallest class changing its prediction moves the surface baseline's macro-F1 by $\rho \in [0.0003, 0.0020]$. The probe is scored on a held-out portion of the target fold and its ρ is larger, $[0.0016, 0.0132]$. Every transfer figure in Table 1 clears its own ρ by more than an order of magnitude. The per-class ablations of Section 5 do not, and we read them only as an upper bound.

What is compared. The *surface baseline* is a character n-gram classifier over the context around an occurrence, with every instance of the variable's own name replaced by a placeholder, fitted on the source language and applied to the target. It involves no model. Its score is the strongest of three masked-context variants per cell, and fixing any single variant in advance gives a *larger* boundary drop (-0.173 to -0.276 against the -0.166 we report), so the reported figure is the conservative choice. The *probe* is a logistic regression on residual-stream activations pooled over the occurrence, with the layer chosen on source-language validation data.

The two are not given the same input. The probe pools over the identifier's own tokens and therefore reads the name. The baseline is forbidden from reading it. We report the identifier alone as a third model-free row of Table 1, so the asymmetry is on the page.

We compare three models: Qwen2.5-Coder-1.5B; Qwen2.5-1.5B, the general-purpose base model it is continued from, sharing its architecture, scale, and tokenizer but not its code-specialisation stage; and an untrained network with that architecture and randomly initialised weights.

A provenance confound. XLCoST distributes Python through a formatted mirror and JavaScript and PHP through a detokenized one, so the lineage boundary is also a formatting boundary. It does not bite: the character analyser collapses whitespace runs before extracting n-grams, so line structure never becomes a feature, and normalising whitespace on both sides moves transfer by 0.0000 on all three decisive pairs (Appendix C).

3 The medium is encoded, and organised by lineage

It is worth first establishing that the model represents the medium at all, and being careful about what that representation is of. Applying PCA to last-token residual activations over seven XLCoST languages, language identity is linearly recoverable from the embedding layer onward: PC1 correlates with a static/dynamic typing label at $r = +0.883$ at layer 0, peaks at $|r| = 0.941$ at layer 4 carrying 37% of the variance, stays above 0.86 through layer 20 apart from a dip to 0.83 at layer 6, and decays to 0.576 by layer 28 (Appendix D).

Table 1: Cross-lingual role transfer, macro-F1 averaged over three roles and the ordered pairs in each group. The two model-free rows differ in what they may read: the first masks the variable's name, the second uses nothing else. The probe pools over the identifier's tokens and so reads the name, which is why the identifier-alone row is reported beside it. The model-free rows track the lineage boundary. The trained probes do not, and the untrained network is flat only in the sense of being uniformly poor.

	JavaScript↔PHP	pairs involving Python	change
Surface n-gram, name masked (no model)	0.952	0.785	-0.166
Identifier alone (no model)	0.867	0.810	-0.056
Probe, Qwen2.5-Coder-1.5B	0.893	0.887	-0.006
Probe, Qwen2.5-1.5B (base)	0.899	0.888	-0.011
Probe, untrained (same arch.)	0.590	0.568	-0.021

The typing axis is not privileged. It is tempting to read that as the model having internalised type discipline, and a control rules it out. We relabelled the same activations under every alternative 4-versus-3 partition of the seven languages ($\binom{7}{4} = 35$ groupings, minus the real one, so 34 controls) and probed each. The true static/dynamic split reaches best-layer F1 1.000. The arbitrary partitions average 0.996, and 5 of the 34 match or exceed the real split. Any grouping of these languages is near-perfectly decodable, because what the model encodes is language *identity*. A partition is then recoverable as a union of identities, whether or not it corresponds to a linguistic property.

What survives the control is the arrangement. Individual languages occupy distinct, stable regions, and the two pairs that sit closest are the two that are historically closest: C beside C++, Java beside C#. The medium is represented sharply, and at least locally it is organised by descent. That is what makes the next section's question worth asking.

4 Role transfer across the lineage boundary

Table 1 splits the six ordered pairs by whether Python, the one language in the matched triangle outside the C family, is involved.

The surface is constrained by the medium. Between JavaScript and PHP the n-gram baseline reaches 0.952 with no model and without seeing the variable's name. Across the Python boundary it loses 0.166. The extreme cell is iterator from PHP to Python, falling to 0.576 against 0.965 from PHP to JavaScript: same source language, same feature family, refit on each pair's own matched set (208 problems against 248). The probe on those cells barely moves (0.865 and 0.851), so the collapse is not an artefact of the smaller set.

The representation is not. The same split costs the code model's probe 0.006. Across all 18 cells the probe's scores are far less dispersed than the masked baseline's (SD 0.028 against 0.100). The probe does not uniformly beat the baseline. It loses all six cells of the closely related pair, where n-grams have already saturated, and that is the point. The model contributes nothing where surface similarity is high and a great deal where it is low.

Two controls decide what this means. A flat curve is worthless if the probe is reading the input rather than the model, and worthless again if it is flat because it is uniformly poor. The untrained network settles both. It reaches 0.575 overall against 0.889 and 0.892 for the trained models, gaps of 0.314 and 0.316 on average, positive in *all eighteen* cells for both, with a minimum per-cell lift of 0.126. the probes are reading something the networks learned. It is also flat in group mean, dropping 0.021, though its per-cell scores range from 0.417 to 0.780, so what is flat is the average and not the network. That is exactly why flatness alone proves nothing: a network that separates the roles poorly everywhere is close to indifferent to where it fails. The trained models also give up far less against their own in-domain ceilings (0.045 and 0.035) than the untrained one does (0.199). What distinguishes the trained models is being flat *and* high. The untrained network is not at chance either. It clears its own shuffled-label control by 0.111, as random projections of token identity should, so 0.575 rather than 0.5 is the floor a representational claim must clear.

Code specialisation is not the cause. The base model of identical architecture, scale, and tokenizer is nearly as invariant: it moves -0.011 against the code model's -0.006, both an order of magnitude below the surface baseline's -0.166. Whatever makes the role representation insensitive to the medium is not the code-specialisation corpus. No genuinely code-free model exists in this family, so the contrast isolates that stage, not code exposure.

5 Why the surface fails: form–meaning realignment, not missing forms

Calling the drop typological names it without explaining it. Because the baseline fits its vectoriser on the source and only transforms the target, an n-gram the target never produces contributes exactly zero, which makes the mechanism measurable.

The obvious explanation is wrong. Weighting each feature by $|\beta_j|$ times its mean tf-idf, roughly 70% of the source classifier's discriminative mass is realised in *every* target, including the worst (0.697 for php→python against 0.735 for php→javascript). Surviving mass shows no detectable relationship with transfer ($r = +0.09$, 95% CI $[-0.78, +0.84]$ over six pairs); more directly, it stays inside a 0.70–0.86 band while F1 spans 0.58–0.97. The cues are present.

What predicts transfer is whether they mean the same thing. The failure has the shape of a false-friends effect in the classifier's feature space: the discriminative forms survive into the target, but their mapping to roles reverses, and it is the reversed share rather than the missing share that predicts transfer. Fitting a second classifier on the target labels in the same feature space, the share of target-realised mass whose sign flips between the two tracks transfer strongly ($r = -0.98$, CI $[-1.00, -0.79]$; -0.91 under a source-weighted variant): 5–7% between JavaScript and PHP against 18–37% across the Python boundary, with no overlap between the groups.

The realignment is distributed rather than localised, which is not consistent with the reading "typological" invites. For the iterator role and the two PHP-source pairs that bracket the entire 18-cell spread, masking an orthographic character class on both sides (terminators, braces, brackets, parentheses, or operators) moves transfer by at most 0.021 against a 0.39 gap (Table 6). none of these deltas is large relative to its own resolution, which is the point. Indentation and keywords are not ablated, so this bounds punctuation, not syntax in general. We report the aggregate only: single coefficients of a regularised model over correlated character n-grams are not interpretable one at a time.

6 Discussion

Read against the design-axis premise, our results split it in two. The medium demonstrably constrains surface generalisation: a strong lexical model loses a sixth of its performance at a lineage boundary, and the model encodes language identity sharply, placing historically adjacent languages together (Section 3). But the medium leaves the learned role representation nearly untouched, and it does so in a network without code-specialised pretraining. A behavioural evaluation reporting one cross-lingual accuracy would average these into a single degraded number and attribute it to the language, when the language is doing all of its work at the surface.

Limitations. The matched triangle instantiates "closely related" by one pair and the boundary by one language. The axis has two points on it and no gradient, and Appendix A shows XLCoST admits no wider version. Both trained models are 1.5B parameters from one family, so nothing here speaks to scale, and that family has no code-free member.

Role labels come from an AST for Python and regular expressions for JavaScript and PHP, and the paths do not carve *iterator* identically: a C-style counter that also accumulates is dropped as multi-role in JavaScript and PHP but kept as an iterator in Python, so the iterator cells mix the representational effect with an extractor-definition difference. Accumulator and index_key, whose extractors agree far better across the boundary, show the same direction (0.782 and 0.839 php→python against 0.954 and 0.964 php→javascript). Section 5 runs on iterator alone, its ablation on two of six pairs, and its correlations on six points. The probe is correlational. The causal interventions of Appendix E cover a different role and were not run on these cells.

References

Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K. Reddy. XLCoST: A benchmark dataset for cross-lingual code intelligence. *arXiv preprint arXiv:2206.08474*, 2022.

A Pairwise problem overlap in XLCoST

Table 2: Shared problem identifiers between every pair of XLCoST languages, train split. Matched cross-lingual transfer needs a pair to share enough problems to hold out a test fold; only the three bold cells clear 500. No pair outside Python/JavaScript/PHP does, including same-family pairs.

	C++	C#	Java	JavaScript	PHP	Python
C++	—	115	122	75	17	74
C#	115	—	175	75	14	62
Java	122	175	—	85	11	66
JavaScript	75	75	85	—	**1,529**	**2,953**
PHP	17	14	11	**1,529**	—	**1,145**
Python	74	62	66	**2,953**	**1,145**	—

B Full transfer table

Table 3: All eighteen transfer cells for Qwen2.5-Coder-1.5B. *Surface* is the strongest masked-context n-gram feature; *name* uses the variable's name alone; *probe* is the residual-stream probe. Majority and shuffled-label controls sit near chance throughout, and ρ is the macro-F1 movement from one test occurrence changing its prediction.

role	pair	n	surface	name	probe	major.	shuf.	ρ
accumulator	JavaScript→PHP	1,976	0.951	0.897	0.904	0.361	0.470	0.0005
accumulator	JavaScript→Python	3,341	0.813	0.873	0.875	0.415	0.473	0.0004
accumulator	PHP→JavaScript	1,903	0.954	0.897	0.918	0.372	0.490	0.0005
accumulator	PHP→Python	1,377	0.782	0.824	0.833	0.276	0.439	0.0008
accumulator	Python→JavaScript	3,294	0.785	0.874	0.918	0.370	0.504	0.0003
accumulator	Python→PHP	1,478	0.736	0.845	0.874	0.271	0.466	0.0007
index_key	JavaScript→PHP	1,976	0.940	0.834	0.897	0.384	0.478	0.0005
index_key	JavaScript→Python	3,341	0.859	0.787	0.883	0.330	0.497	0.0003
index_key	PHP→JavaScript	1,903	0.964	0.838	0.880	0.376	0.516	0.0005
index_key	PHP→Python	1,377	0.839	0.782	0.854	0.387	0.515	0.0008
index_key	Python→JavaScript	3,294	0.871	0.822	0.915	0.314	0.470	0.0003
index_key	Python→PHP	1,478	0.858	0.838	0.874	0.419	0.459	0.0008
iterator	JavaScript→PHP	1,976	0.937	0.862	0.906	0.448	0.488	0.0008
iterator	JavaScript→Python	3,341	0.774	0.733	0.939	0.444	0.472	0.0005
iterator	PHP→JavaScript	1,903	0.965	0.873	0.851	0.446	0.478	0.0008
iterator	PHP→Python	1,377	0.576	0.788	0.865	0.428	0.462	0.0010
iterator	Python→JavaScript	3,294	0.790	0.769	0.918	0.466	0.484	0.0007
iterator	Python→PHP	1,478	0.741	0.788	0.902	0.475	0.438	0.0020

Table 4: Probe transfer per model, averaged within each group. The untrained network shares Qwen2.5-1.5B's architecture and tokenizer with randomly initialised weights; it is the floor a representational claim must clear, and it is itself flat.

model	close pair	Python pairs	all	vs. untrained
Qwen2.5-1.5B	0.899	0.888	0.892	+0.316
Qwen2.5-Coder-1.5B	0.893	0.887	0.889	+0.314
Qwen2.5-1.5B (untrained)	0.590	0.568	0.575	—

C Transfer mechanism

Table 5: Transfer mechanism for the iterator role. *Surviving* is the share of the source classifier's discriminative mass realised in the target; *flip* is the share of that mass whose sign reverses. Surviving mass is uncorrelated with transfer; flipped mass predicts it almost exactly. Source-weighted variants are given for robustness.

pair	F1	surviving	flip	agree	flip (src-wt.)
PHP→JavaScript	0.965	0.735	0.072	+0.895	0.072
JavaScript→PHP	0.937	0.698	0.052	+0.945	0.068
Python→JavaScript	0.790	0.862	0.178	+0.695	0.211
JavaScript→Python	0.774	0.757	0.232	+0.811	0.241
Python→PHP	0.741	0.696	0.280	+0.528	0.367
PHP→Python	0.576	0.697	0.366	+0.476	0.346

Table 6: Masking an entire syntactic character class on both sides of the transfer. No class accounts for more than 0.021 of the 0.39 gap, so the realignment is distributed rather than carried by block delimiters or statement terminators.

pair	masked class	macro-F1	Δ
PHP→JavaScript	terminator	0.9496	-0.0159
PHP→JavaScript	brace	0.9645	-0.0010
PHP→JavaScript	bracket	0.9711	+0.0057
PHP→JavaScript	paren	0.9609	-0.0045
PHP→JavaScript	operator	0.9441	-0.0213
PHP→Python	terminator	0.5786	+0.0030
PHP→Python	brace	0.5750	-0.0006
PHP→Python	bracket	0.5733	-0.0023
PHP→Python	paren	0.5947	+0.0191
PHP→Python	operator	0.5801	+0.0045

D Language geometry

The typing axis is not privileged. Probing the same activations under every alternative 4-versus-3 partition of the seven languages ($\binom{7}{4} = 35$ groupings minus the real one, so 34 controls) gives best-layer macro-F1 0.9959 on average (min 0.9918, max 1.0000), against 1.0000 for the true static/dynamic split; 5 of the 34 controls match or exceed it. Any grouping of these languages is near-perfectly decodable, so the decodability of the typing split is evidence that language identity is encoded, not that type discipline is. At layer 4, where $|r|$ peaks at 0.941, the remaining components carry almost none of the label: PC2 $r = -0.014$, PC3 $r = +0.026$, PC4 $r = +0.171$.

E Causal interventions

Scope. Causal interventions were run for the *boolean* role over 5 languages (C++, Java, JavaScript, PHP, Python), 3 models, and 3 intervention modes (ablate, patch, steer), giving 375 layer-wise measurements. They are reported here rather than in the main text because they were run *within* languages and on a role outside the three this paper transfers, so they cannot support its central claim. Specificity is $|\text{clean} - \text{intervened}|$ divided by the same quantity for a random-position control: how much of the effect is specific to the variable's own token positions rather than to editing anything at all. Two columns are given because the layer with the largest effect is generally not the layer with the best specificity, and quoting either alone misleads.

F Reproducibility

Every figure in the main text is regenerated from the committed result files by the released pipeline, and a regression test recomputes each of them—including the three correlations of Section 5—from

Table 7: Point-biserial correlation between PC1 of last-token residual activations and a static/dynamic typing label, by layer, with PC1's explained-variance ratio. The sign of r is arbitrary (PCA fixes the axis only up to reflection, and each layer is refit independently); magnitude is what carries meaning.

layer	$\lvert r\rvert$	EVR	layer	$\lvert r\rvert$	EVR
0	0.883	0.481	15	0.894	0.309
1	0.898	0.447	16	0.882	0.303
2	0.914	0.391	17	0.874	0.296
3	0.939	0.373	18	0.873	0.289
4	0.941	0.370	19	0.873	0.299
5	0.862	0.369	20	0.903	0.291
6	0.828	0.361	21	0.886	0.276
7	0.844	0.342	22	0.859	0.265
8	0.896	0.339	23	0.842	0.252
9	0.919	0.333	24	0.812	0.242
10	0.880	0.327	25	0.774	0.253
11	0.893	0.302	26	0.694	0.255
12	0.917	0.292	27	0.732	0.237
13	0.904	0.314	28	0.576	0.242
14	0.898	0.309			

those files rather than quoting them. Section 3 draws on a separate activation-extraction pipeline; its layer table, principal-component correlations, and bucket-control figures are committed under `results/lp4fm/language_geometry/`, and the values quoted are read from those files. No claim in Sections 4–5 depends on it.

Table 8: Causal interventions on boolean-flag occurrences. *Peak-effect spec.* is specificity at the largest-effect layer; *best spec.* is the maximum over layers, with that layer named.

language	mode	model	peak-effect spec.	best spec. (layer)
C++	ablate	qwen2515b	4.5×	31.9× (L4)
C++	ablate	qwen25coder15b	4.4×	76.2× (L20)
C++	ablate	starcoder27b	4.3×	36.5× (L4)
C++	patch	qwen2515b	3.6×	7.9× (L20)
C++	patch	qwen25coder15b	3.6×	6.9× (L8)
C++	patch	starcoder27b	8.1×	10.4× (L16)
C++	steer	qwen2515b	45.0×	169.8× (L4)
C++	steer	qwen25coder15b	598.5×	598.5× (L24)
C++	steer	starcoder27b	6.9×	341.1× (L4)
Java	ablate	qwen2515b	28.5×	86.0× (L12)
Java	ablate	qwen25coder15b	12.3×	62.4× (L12)
Java	ablate	starcoder27b	27.0×	27.0× (L4)
Java	patch	qwen2515b	6.1×	7.7× (L12)
Java	patch	qwen25coder15b	5.9×	7.7× (L12)
Java	patch	starcoder27b	7.5×	8.6× (L12)
Java	steer	qwen2515b	72.1×	849.0× (L8)
Java	steer	qwen25coder15b	71.9×	186.6× (L0)
Java	steer	starcoder27b	252.0×	252.0× (L24)
JavaScript	ablate	qwen2515b	26.7×	26.7× (L16)
JavaScript	ablate	qwen25coder15b	28.1×	116.3× (L24)
JavaScript	ablate	starcoder27b	27.3×	27.3× (L4)
JavaScript	patch	qwen2515b	6.9×	6.9× (L8)
JavaScript	patch	qwen25coder15b	6.6×	7.4× (L12)
JavaScript	patch	starcoder27b	8.1×	8.2× (L16)
JavaScript	steer	qwen2515b	24.5×	76.4× (L8)
JavaScript	steer	qwen25coder15b	65.2×	149.5× (L4)
JavaScript	steer	starcoder27b	11.2×	60.8× (L28)
PHP	ablate	qwen2515b	15.0×	102.4× (L20)
PHP	ablate	qwen25coder15b	26.5×	79.2× (L20)
PHP	ablate	starcoder27b	19.4×	19.7× (L8)
PHP	patch	qwen2515b	7.2×	7.2× (L4)
PHP	patch	qwen25coder15b	4.7×	7.4× (L0)
PHP	patch	starcoder27b	6.3×	9.3× (L16)
PHP	steer	qwen2515b	190.3×	190.3× (L24)
PHP	steer	qwen25coder15b	11.5×	134.0× (L8)
PHP	steer	starcoder27b	8.7×	100.1× (L4)
Python	ablate	qwen2515b	5.8×	36.7× (L24)
Python	ablate	qwen25coder15b	6.8×	33.1× (L16)
Python	ablate	starcoder27b	26.5×	26.5× (L4)
Python	patch	qwen2515b	6.0×	7.2× (L16)
Python	patch	qwen25coder15b	6.7×	7.8× (L16)
Python	patch	starcoder27b	7.9×	9.7× (L12)
Python	steer	qwen2515b	209.1×	827.9× (L16)
Python	steer	qwen25coder15b	1025.8×	1025.8× (L24)
Python	steer	starcoder27b	8.6×	55.5× (L4)